DATA-260 Homework 9 — Redis Counter, Semantic Cache, Threshold Economics, Self-Improving Agents

Xu Wang_018319275

GitHub repository: https://github.com/xuwang-goldenwater/data260-9275 (code: code/cache_app/, code/self_improve/; evidence: reports/hw09/)

Section 0 — Personal Configuration

Value	Definition	My value
SID4	Last four digits of SJSU ID	9275
PORT_BASE	8000 + (SID4 mod 900)	8275
PREFIX	"s" + SID4	s9275 (every Redis key starts with s9275:)
SEED	SID4	9275
VERIFY_SEED	260000 + SID4	269275
DOMAIN_ID	SID4 mod 8	3 — Grocery supply and recall notices

Item	Value
Hardware	MacBook Pro, Apple M1, 8 GB RAM; Docker Desktop
Redis	redis/redis-stack-server:7.4.0-v1 (Redis + RediSearch vector index), 127.0.0.1:6379
Database	MySQL 8.0, s9275_rel (recall_notices + firms, 5002 rows)
Embedding	sentence-transformers/all-MiniLM-L6-v2, CPU, 384 dims, unit length
Model	qwen2.5:3b via Ollama (temperature 0, seed 9275) through src/model_client.py
Builds on	HW8: Kafka + Planner -> Writer -> Reviewer pipeline; HW5 MySQLRepo / get_notice

Part 1 — Redis View Counter

Entity: recall notice. code/sql/hw09_migrate.sql adds recall_notices.view_count. The one new method, RecallCache.increment_recall_notice_views(notice_id), increments a Redis key with INCR on every access and copies the count to the database row every 10th access.

code/cache_app/recall_cache.py — increment_recall_notice_views()

```
    def increment_recall_notice_views(self, notice_id: int) -> int:
        """Count one view of a recall notice and return the running total."""
        # 1. Increment in Redis. INCR is atomic: two requests at the same time
        #    get two different numbers, so no view is lost.
        key = f"{PREFIX}:views:recall_notice:{notice_id}"
        count = self.redis_client.incr(key)

        # 2. Every 10th view, copy the count to the database row.
        #    updated_at = updated_at keeps the column's ON UPDATE CURRENT_TIMESTAMP
        #    from firing: a view is not a change to the recall notice itself.
        self.last_synced = False
        if count % SYNC_EVERY == 0:
            with self.Session() as db:
                db.execute(
                    text("UPDATE recall_notices SET view_count = :c, updated_at = updated_at "
                         "WHERE id = :id"),
                    {"c": count, "id": notice_id},
                )
                db.commit()
            self.last_synced = True
            log.info("Synced views to database: notice %s view_count=%s", notice_id, count)

        # 3. Return the current count.
        return count
```

```
Xu Wang | DATA 260 HW9 Part 1 | PREFIX s9275

=== Testing Counter === (recall_notices.id = 1)

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
  [log] Synced views to database: notice 1 view_count=10
Access #10: Total = 10
✓ Synced views to database
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
  [log] Synced views to database: notice 1 view_count=20
Access #20: Total = 20
✓ Synced views to database
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25

Redis  s9275:views:recall_notice:1 = 25
MySQL  recall_notices.view_count (id=1)   = 20
```

Screenshot 1.1 — python code/cache_app/recall_cache.py — Access #1 … #25, sync at 10 and 20, Redis 25 vs MySQL 20

After 25 accesses Redis holds 25 and MySQL holds 20: the database received 2 writes instead of 25 and lags by up to 9 views. INCR is atomic, so concurrent views are not lost. updated_at = updated_at keeps the column's ON UPDATE timestamp from firing, because a view is not a change to the notice (this matters for Part 3.5).

Part 2 — Semantic Response Cache

SemanticCache.ask() is the ingress in front of the HW8 pipeline. Hit: the stored final answer is returned and no agent runs. Miss: the recall record named in the question is looked up (HW5 MySQLRepo.get) and sent with the question as facts through Planner -> Writer -> Reviewer over Kafka; an approved answer is stored. Each entry is a Redis hash s9275:semcache:<sha1> with query, embedding (384 x float32), answer, notice_id, data_version, created_at; the index s9275:semcache_idx is FLAT with COSINE distance (similarity = 1 − distance).

code/cache_app/semantic_cache.py — ask() (hit / guard / stale / miss)

```
    def ask(self, question: str) -> dict:
```

```
        t0 = time.perf_counter()
        vec = self.embed(question)
        t_embed = time.perf_counter()
        best = self.lookup(vec)
        t_search = time.perf_counter()
        sim = best["similarity"] if best else None
        timings = {"embed_ms": (t_embed - t0) * 1000, "search_ms": (t_search - t_embed) * 1000}

        if best and sim >= self.threshold and self.recall_guard \
                and not recall_guard_ok(question, best["query"]):
            log.info("CACHE GUARD sim=%.4f but recall number differs from %r -> miss",
                     sim, best["query"])
            best = None                       # not allowed to reuse; fall through to the agents
        if best and sim >= self.threshold:
            stale = VERSION_CHECK and best["data_version"] !=
self.current_version(best["notice_id"])
            if not stale:
                total = (time.perf_counter() - t0) * 1000
                log.info("CACHE HIT  sim=%.4f  matched=%r", sim, best["query"])
                return self._result(question, best["answer"], True, sim, total, timings,
                                    matched=best["query"], notice_id=best["notice_id"])
            self.r.delete(best["key"])
            log.info("CACHE STALE sim=%.4f  v%s != v%s  -> entry deleted, treated as miss",
                     sim, best["data_version"], self.current_version(best["notice_id"]))

        log.info("CACHE MISS best_sim=%s -> HW8 agent pipeline",
                 "none" if sim is None else f"{sim:.4f}")
        facts, notice_id = self.facts_for(question)
        version = self.current_version(notice_id)
        t_agent = time.perf_counter()
        final = self.run_agent(question, facts)
        timings["agent_ms"] = (time.perf_counter() - t_agent) * 1000
        answer = final.get("answer", "")
        if final.get("status") == "approved":       # never cache an answer the Reviewer rejected
            self.store(question, vec, answer, notice_id, version)
        total = (time.perf_counter() - t0) * 1000
        return self._result(question, answer, False, sim, total, timings,
                            notice_id=notice_id, status=final.get("status"))
```

```
Xu Wang | DATA 260 HW9 Part 2 | semantic cache demo | PREFIX s9275
Loading weights: 100%|██████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 4801.43it/s]
14:16:42 created index s9275:semcache_idx on s9275:semcache:*

--- Q01 [new] How many units were affected by recall F-0001-2025?
14:16:42 CACHE MISS best_sim=none -> HW8 agent pipeline
    is_cached=False  similarity=  -     latency=16885.4 ms
    answer: Recall F-0001-2025 affected 8019 units of Blue Heron Enoki Mushrooms, 32 oz tub. This recall was due to an undeclared allergen, and the firm involved is Harbor

--- Q02 [duplicate] How many units were affected by recall F-0001-2025?
14:16:59 CACHE HIT  sim=1.0000  matched='How many units were affected by recall F-0001-2025?'
    is_cached=True  similarity=1.0000  latency=252.1 ms
    answer: Recall F-0001-2025 affected 8019 units of Blue Heron Enoki Mushrooms, 32 oz tub. This recall was due to an undeclared allergen, and the firm involved is Harbor

--- Q03 [paraphrase] For recall F-0001-2025, what was the number of units affected?
14:16:59 CACHE HIT  sim=0.9717  matched='How many units were affected by recall F-0001-2025?'
    is_cached=True  similarity=0.9717  latency=21.5 ms
    answer: Recall F-0001-2025 affected 8019 units of Blue Heron Enoki Mushrooms, 32 oz tub. This recall was due to an undeclared allergen, and the firm involved is Harbor

--- Q04 [new] Which firm issued recall F-0002-2021?
14:16:59 CACHE MISS best_sim=0.7265 -> HW8 agent pipeline
    is_cached=False  similarity=0.7265  latency=24641.5 ms
    answer: The firm that issued recall F-0002-2021 is Maple Snacks Co., based on the information available. This recall was for Green Leaf Almond Granola, 16 oz, due to an

--- Q05 [paraphrase] Which company is behind recall F-0002-2021?
14:17:24 CACHE HIT  sim=0.9356  matched='Which firm issued recall F-0002-2021?'
    is_cached=True  similarity=0.9356  latency=294.7 ms
    answer: The firm that issued recall F-0002-2021 is Maple Snacks Co., based on the information available. This recall was for Green Leaf Almond Granola, 16 oz, due to an

--- Q06 [new] What should a grocery store do when it receives a recall notice for peanut butter that may contain undeclared milk?
14:17:24 CACHE MISS best_sim=0.4278 -> HW8 agent pipeline
    is_cached=False  similarity=0.4278  latency=18268.6 ms
    answer: Upon receiving a recall notice for peanut butter potentially containing undeclared milk, a grocery store should immediately assess and document all affected pro

--- Q07 [paraphrase] How should a grocery store respond to a recall notice for peanut butter that may contain undeclared milk?
14:17:43 CACHE HIT  sim=0.9783  matched='What should a grocery store do when it receives a recall notice for peanut butter that may contain undeclared milk?'
    is_cached=True  similarity=0.9783  latency=287.9 ms
    answer: Upon receiving a recall notice for peanut butter potentially containing undeclared milk, a grocery store should immediately assess and document all affected pro

--- Q08 [duplicate] Which firm issued recall F-0002-2021?
14:17:43 CACHE HIT  sim=1.0000  matched='Which firm issued recall F-0002-2021?'
    is_cached=True  similarity=1.0000  latency=21.3 ms
    answer: The firm that issued recall F-0002-2021 is Maple Snacks Co., based on the information available. This recall was for Green Leaf Almond Granola, 16 oz, due to an

--- Q09 [new] When was recall F-0003-2023 issued?
14:17:43 CACHE HIT  sim=0.8518  matched='Which firm issued recall F-0002-2021?'
    is_cached=True  similarity=0.8518  latency=13.6 ms
    answer: The firm that issued recall F-0002-2021 is Maple Snacks Co., based on the information available. This recall was for Green Leaf Almond Granola, 16 oz, due to an

--- Q10 [duplicate] What should a grocery store do when it receives a recall notice for peanut butter that may contain undeclared milk?
14:17:43 CACHE HIT  sim=1.0000  matched='What should a grocery store do when it receives a recall notice for peanut butter that may contain undeclared milk?'
```

Screenshot 2.1 — python code/cache_app/part2_demo.py — CACHE HIT (with similarity) and CACHE MISS log lines

```
Xu Wang | DATA 260 HW9 | saved evidence: part2 | PREFIX s9275
(saved run 2026-11-06T14:18:24, reports/hw09/raw/part2_demo.json)

=== Summary (threshold 0.85) ===
  #  kind       cached     sim  latency ms
  1  new        False        -     16885.4
  2  duplicate  True    1.0000       252.1
  3  paraphrase True    0.9717        21.5
  4  new        False   0.7265     24641.5
  5  paraphrase True    0.9356       294.7
  6  new        False   0.4278     18268.6
  7  paraphrase True    0.9783       287.9
  8  duplicate  True    1.0000        21.3
  9  new        True    0.8518        13.6
 10  duplicate  True    1.0000        17.1
 11  new        False   0.5655     17896.9
 12  paraphrase False   0.7500     23077.5

Cache hit rate : 7/12 = 58.3%
Avg latency    : cached 129.7 ms  |  not cached 20154.0 ms
```

Screenshot 2.2 — Part 2 summary: 12 rows, cache hit rate, average latency cached vs not cached (reprinted from raw/part2_demo.json)

#	Kind	is_cached	Similarity	Latency (ms)
1	new	False	-	16885.4
2	duplicate	True	1.0000	252.1
3	paraphrase	True	0.9717	21.5
4	new	False	0.7265	24641.5
5	paraphrase	True	0.9356	294.7
6	new	False	0.4278	18268.6
7	paraphrase	True	0.9783	287.9
8	duplicate	True	1.0000	21.3
9	new	True (false hit)	0.8518	13.6
10	duplicate	True	1.0000	17.1
11	new	False	0.5655	17896.9
12	paraphrase	False	0.7500	23077.5

Cache hit rate: 7/12 = 58.3%. **Average latency:** cached 129.7 ms (n = 7, median 21.5 ms) vs not cached 20,154.0 ms (n = 5).

- Reported as measured. The Redis search itself took 1.2–3.0 ms on every hit; the rest is embedding. The three 250–295 ms hits each came right after a 17–25 s model call.
- Q9 "When was recall F-0003-2023 issued?" hit "Which firm issued recall F-0002-2021?" at 0.8518 and returned another recall's firm — a false hit at the default 0.85, examined in Part 3.
- Found while testing: the HW8 Reviewer (qwen2.5:3b) rejected a correct answer twice because it could not see the database facts. Fix: the Reviewer receives the same facts as the Writer plus a rule that rejects numbers not in the facts (code/kafka_agents/reviewer.py).

Part 3 — Threshold Economics and Stale Answers

3.1 Labeled test set

reports/hw09/cache_cases.yaml: 10 seeds (6 record questions, one field each, from real s9275_rel rows; 4 general questions) and 40 cases — 10 exact duplicates and 10 paraphrases (expect a hit on their own seed), 10 superficially similar questions with a different correct answer and 10 unrelated questions (expect a miss). Definitions were fixed in the file header before any run: a false hit is a similar/unrelated case that hits, or an exact/paraphrase case that hits a different seed; false-hit rate = false hits / hits. Committed in 4188961, before the first sweep result (93e295a), never edited — both checked from git history by verify_hw09.py.

cache_cases.yaml — excerpt

```
  - {id: S1, query: "How many units were affected by recall F-0001-2025?",
     recall_number: F-0001-2025, field: units_affected, value: 8019}
  ...
  - {id: M03, kind: similar, seed: S3, expect: miss, query: "What is the recall date of F-0001-
2025?",
     why: "same wording as S3, recall number of S1: 2025-06-23, not 2023-12-29"}
```

3.2 Threshold sweep

prepare: the 10 seeds were answered by the agents and cached (all approved on the first try); each of the other 30 cases ran through the agents once, not stored, giving its correct answer and real miss latency (mean 18.4 s). sweep: the cache is frozen at the 10 seeds; at each threshold every case is embedded and searched for real, and a miss adds that case's measured agent time (the agents were not re-run 5 × 40 times).

```
Xu Wang | DATA 260 HW9 Part 3.2 | threshold sweep | PREFIX s9275
Loading weights: 100%|██████████████████| 103/103 [00:00<00:00, 5609.98it/s]

=== 1. seeds -> agents -> cache ===
S1   try 1  approved    30098 ms  How many units were affected by recall F-0001-2025?
S2   try 1  approved    26035 ms  Which firm issued recall F-0002-2021?
S3   try 1  approved    24503 ms  What is the recall date of F-0003-2023?
S4   try 1  approved    21544 ms  What product was recalled under F-4393-2026?
S5   try 1  approved    23405 ms  What is the recall category of F-2015-2026?
S6   try 1  approved    23699 ms  In which state is the firm for recall F-4529-2026 located?
S7   try 1  approved    26720 ms  What should a grocery store do when it receives a recall notice for peanut butter that may contain undeclared milk?
S8   try 1  approved    19246 ms  How should a grocery store dispose of recalled products?
S9   try 1  approved    21145 ms  How should a store handle refunds for customers who return recalled items?
S10  try 1  approved    14889 ms  How should a grocery store notify customers about a product recall?

=== 2. non-exact cases -> agents (not stored) ===
P01  approved    18382 ms  What was the number of units affected by recall F-0001-2025?
P02  approved    19202 ms  Which company is responsible for recall F-0002-2021?
P03  approved    20121 ms  On what date was recall F-0003-2023 announced?
P04  approved    20916 ms  Which product does recall F-4393-2026 cover?
P05  approved    21626 ms  Why was the product in recall F-2015-2026 recalled?
P06  rejected    19745 ms  Where is the company behind recall F-4529-2026 based?
P07  approved    16547 ms  How should a grocery store respond to a recall notice for peanut butte
P08  approved    14838 ms  What is the proper way for a grocery store to get rid of recalled prod
P09  approved    20176 ms  How should a store process refunds when customers bring back recalled
P10  approved    13038 ms  What is the best way for a grocery store to tell customers about a pro
M01  approved    19257 ms  How many units were affected by recall F-0002-2021?
M02  approved    18538 ms  Which firm issued recall F-0003-2023?
M03  approved    23080 ms  What is the recall date of F-0001-2025?
M04  approved    24368 ms  What product was recalled under F-4906-2026?
M05  approved    18446 ms  What is the recall category of F-1867-2026?
M06  approved    15172 ms  In which state is the firm for recall F-2320-2026 located?
M07  approved    15158 ms  What should a consumer do if they bought peanut butter that may contai
M08  approved    14047 ms  How should a grocery store hold recalled products until the supplier c
M09  approved    15829 ms  How should a store get reimbursed by the supplier for recalled items i
M10  approved    19443 ms  How should a food company notify the FDA about a product recall?
U01  rejected    11611 ms  What are the store's opening hours on Sunday?
U02  approved    11350 ms  How do I reset my account password?
U03  approved     9090 ms  What is the capital of France?
U04  rejected    13368 ms  How long should I boil an egg?
U05  rejected    12554 ms  What is the weather in San Jose today?
U06  approved    13315 ms  How do I make a sourdough starter at home?
U07  approved    12506 ms  What temperature should a grocery store freezer be set to?
U08  rejected    15570 ms  How do I apply for a cashier job at a grocery store?
U09  rejected    19486 ms  What is the shelf life of an opened jar of tahini?
U10  rejected    16384 ms  How many recall notices were issued in total in 2024?

cache now holds 10 entries (expected 10); saved reports/hw09/raw/sweep_prepare.json
```

Screenshot 3.1 — threshold_sweep.py prepare — 10 seeds approved, 30 cases answered, cache holds 10 entries

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	75.0% (30/40)	36.7% (11/30)	3394.9	230
0.80	70.0% (28/40)	35.7% (10/28)	4300.7	212
0.85	62.5% (25/40)	28.0% (7/25)	5703.9	155
0.90	57.5% (23/40)	21.7% (5/23)	6538.8	117
0.95	40.0% (16/40)	18.8% (3/16)	9693.2	84

```
Xu Wang | DATA 260 HW9 | saved evidence: sweep | PREFIX s9275
(saved run 2026-11-06T15:07:33, reports/hw09/raw/sweep_summary.json + sweep_cases.csv)

=== Threshold sweep (40 cases, cache frozen at 10 seeds) ===
thresh  hits  correct  false  missed  hit rate  false-hit rate  mean latency ms
  0.75    30       19     11       0     75.0%           36.7%           3394.9
  0.80    28       18     10       1     70.0%           35.7%           4300.7
  0.85    25       18      7       2     62.5%           28.0%           5703.9
  0.90    23       18      5       2     57.5%           21.7%           6538.8
  0.95    16       13      3       7     40.0%           18.8%           9693.2

FALSE HITS:
  t=0.75  P05 (paraphrase) -> S4  sim 0.8436  Why was the product in recall F-2015-2026 recalled?
  t=0.75  M01 (similar) -> S1  sim 0.9742  How many units were affected by recall F-0002-2021?
  t=0.75  M02 (similar) -> S2  sim 0.9505  Which firm issued recall F-0003-2023?
  t=0.75  M03 (similar) -> S3  sim 0.9758  What is the recall date of F-0001-2025?
  t=0.75  M04 (similar) -> S4  sim 0.9335  What product was recalled under F-4906-2026?
  t=0.75  M05 (similar) -> S5  sim 0.8370  What is the recall category of F-1867-2026?
  t=0.75  M06 (similar) -> S6  sim 0.9354  In which state is the firm for recall F-2320-2026 located?
  t=0.75  M07 (similar) -> S7  sim 0.7866  What should a consumer do if they bought peanut butter that may contain undeclared milk?
  t=0.75  M08 (similar) -> S8  sim 0.8774  How should a grocery store hold recalled products until the supplier collects them?
  t=0.75  M09 (similar) -> S9  sim 0.8295  How should a store get reimbursed by the supplier for recalled items it removed?
  t=0.75  M10 (similar) -> S10  sim 0.8694  How should a food company notify the FDA about a product recall?
  t=0.80  P05 (paraphrase) -> S4  sim 0.8436  Why was the product in recall F-2015-2026 recalled?
  t=0.80  M01 (similar) -> S1  sim 0.9742  How many units were affected by recall F-0002-2021?
  t=0.80  M02 (similar) -> S2  sim 0.9505  Which firm issued recall F-0003-2023?
  t=0.80  M03 (similar) -> S3  sim 0.9758  What is the recall date of F-0001-2025?
  t=0.80  M04 (similar) -> S4  sim 0.9335  What product was recalled under F-4906-2026?
  t=0.80  M05 (similar) -> S5  sim 0.8370  What is the recall category of F-1867-2026?
  t=0.80  M06 (similar) -> S6  sim 0.9354  In which state is the firm for recall F-2320-2026 located?
  t=0.80  M08 (similar) -> S8  sim 0.8774  How should a grocery store hold recalled products until the supplier collects them?
  t=0.80  M09 (similar) -> S9  sim 0.8295  How should a store get reimbursed by the supplier for recalled items it removed?
  t=0.80  M10 (similar) -> S10  sim 0.8694  How should a food company notify the FDA about a product recall?
  t=0.85  M01 (similar) -> S1  sim 0.9742  How many units were affected by recall F-0002-2021?
  t=0.85  M02 (similar) -> S2  sim 0.9505  Which firm issued recall F-0003-2023?
  t=0.85  M03 (similar) -> S3  sim 0.9758  What is the recall date of F-0001-2025?
  t=0.85  M04 (similar) -> S4  sim 0.9335  What product was recalled under F-4906-2026?
  t=0.85  M06 (similar) -> S6  sim 0.9354  In which state is the firm for recall F-2320-2026 located?
  t=0.85  M08 (similar) -> S8  sim 0.8774  How should a grocery store hold recalled products until the supplier collects them?
  t=0.85  M10 (similar) -> S10  sim 0.8694  How should a food company notify the FDA about a product recall?
  t=0.90  M01 (similar) -> S1  sim 0.9742  How many units were affected by recall F-0002-2021?
  t=0.90  M02 (similar) -> S2  sim 0.9505  Which firm issued recall F-0003-2023?
  t=0.90  M03 (similar) -> S3  sim 0.9758  What is the recall date of F-0001-2025?
  t=0.90  M04 (similar) -> S4  sim 0.9335  What product was recalled under F-4906-2026?
  t=0.90  M06 (similar) -> S6  sim 0.9354  In which state is the firm for recall F-2320-2026 located?
  t=0.95  M01 (similar) -> S1  sim 0.9742  How many units were affected by recall F-0002-2021?
  t=0.95  M02 (similar) -> S2  sim 0.9505  Which firm issued recall F-0003-2023?
  t=0.95  M03 (similar) -> S3  sim 0.9758  What is the recall date of F-0001-2025?
```

Screenshot 3.2 — threshold sweep table and FALSE HITS list (reprinted from raw/sweep_summary.json and sweep_cases.csv)

The superficially similar record questions (same wording, other recall number) scored 0.93–0.98 — higher than some true paraphrases (P04 0.77). No threshold separates them: at 0.95 three wrong recalls are still served while 7 correct hits are already lost. The general-question look-alikes scored 0.79–0.88, so 0.90 removes them.

3.3 One false hit in full

	M03 — a false hit at every threshold, 0.75 to 0.95
Cached query (S3)	What is the recall date of F-0003-2023?
Incoming query (M03)	What is the recall date of F-0001-2025?
Similarity	0.9758
Cached answer returned	The recall date for F-0003-2023 is 2023-12-29. This specific date is mentioned in the recall notice for the Sunrise Valley Tahini, 6-pack batch affected by mislabeling from Harbor Mills Inc. in Georgia.
Correct answer	2025-06-23 (F-0001-2025, Blue Heron Enoki Mushrooms, 32 oz tub, Harbor Snacks Co.). The agents' own answer to M03: "The recall date for F-0001-2025 is 2025-06-23. This specific recall was initiated for Blue Heron Enoki Mushroo…"

Likely embedding confusion: the two questions are identical except for the recall number. MiniLM breaks "F-0003-2023" and "F-0001-2025" into short word pieces that differ in only two pieces, and mean pooling averages them with a dozen identical tokens. The model was trained to match meaning, and both sentences mean "the date of a recall"; the identifier that decides the answer carries almost no weight in the vector.

3.4 Cost model and threshold choice

- **Cache-miss cost = 1:** one full pipeline run — 3 qwen2.5:3b calls, 18.4 s of local model time on average. Every miss pays it.
- **False-hit cost = 20:** a store receives facts for the wrong recall (date, firm, units, product). It may pull the wrong product or keep selling the recalled one: a support ticket plus a food-safety exposure, far more than 20 s of compute. A correct hit costs 0.

Total cost = misses × 1 + false hits × 20 (last column of the table in 3.2). With the threshold alone, 0.95 is cheapest (84) and remains the choice for any false-hit cost ≥ 4 — but it still serves 3 wrong recalls. I added one control: a recall-number guard (a hit is allowed only if both questions name the same recall number; general questions with no number are unaffected). Same cases, same measured similarities and agent times:

Threshold + guard	Hit rate	False-hit rate	Mean latency (ms)	False hits	Total cost
0.75	57.5%	17.4%	6907.1	4	97
0.80	52.5%	14.3%	7812.9	3	79
0.85	50.0%	10.0%	8214.3	2	60
0.90	45.0%	0.0%	9049.1	0	**22**
0.95	32.5%	0.0%	11215.1	0	27

Shipped: threshold 0.90 + recall-number guard (18 correct hits, 0 false hits, cost 22). The threshold handles closeness of meaning; the guard handles the identity of the record, which the embedding cannot see. 0.90 rather than 0.95 because the guard already removes the dangerous hits, and 0.95 would throw away 5 more correct ones.

code/cache_app/semantic_cache.py — recall_guard_ok()

```
def recall_guard_ok(incoming: str, cached: str) -> bool:
    """Part 3.4: a hit may only reuse an answer about the SAME recall number.
    The sweep showed MiniLM scores 'units of F-0001-2025' vs 'units of F-0002-2021'
    at 0.97, above real paraphrases, so the threshold cannot separate them.
    Both questions without a recall number -> no constraint (general questions)."""
    a = {m.upper() for m in RECALL_NUMBER.findall(incoming)}
    b = {m.upper() for m in RECALL_NUMBER.findall(cached)}
    return a == b
```

```
Xu Wang | DATA 260 HW9 Part 3.4 | cost model | PREFIX s9275

MISS_COST = 1   FALSE_HIT_COST = 20

=== threshold only (required table) ===
thresh  hit rate  false-hit rate  mean latency ms  misses  false  total cost
  0.75     75.0%           36.7%           3394.9      10     11         230
  0.80     70.0%           35.7%           4300.8      12     10         212
  0.85     62.5%           28.0%           5703.9      15      7         155
  0.90     57.5%           21.7%           6538.8      17      5         117
  0.95     40.0%           18.8%           9693.2      24      3          84

=== threshold + recall guard ===
thresh  hit rate  false-hit rate  mean latency ms  misses  false  total cost
  0.75     57.5%           17.4%           6907.1      17      4          97
  0.80     52.5%           14.3%           7812.9      19      3          79
  0.85     50.0%           10.0%           8214.3      20      2          60
  0.90     45.0%            0.0%           9049.1      22      0          22
  0.95     32.5%            0.0%          11215.1      27      0          27

Lowest total cost: threshold 0.90 + recall guard (cost 22, 18 correct hits, 0 false hits)

Sensitivity, threshold only - cheapest threshold for each false-hit cost:
  FALSE_HIT_COST=  1 -> 0.75
  FALSE_HIT_COST=  2 -> 0.90
  FALSE_HIT_COST=  3 -> 0.90
  FALSE_HIT_COST=  4 -> 0.95
  FALSE_HIT_COST=  5 -> 0.95
  FALSE_HIT_COST= 10 -> 0.95
  FALSE_HIT_COST= 20 -> 0.95
  FALSE_HIT_COST= 50 -> 0.95

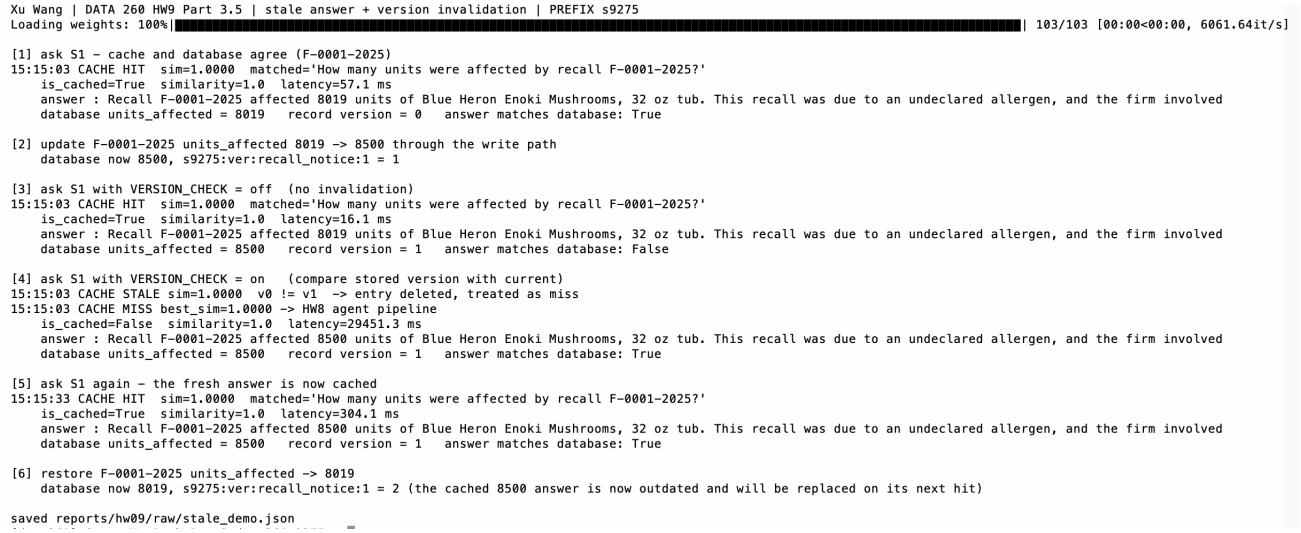
saved reports/hw09/raw/cost_model.json
```

Screenshot 3.4 — python code/cache_app/cost_model.py — threshold-only and threshold + guard tables, sensitivity

3.5 Stale answers and invalidation

Record F-0001-2025 (recall_notices.id = 1); cached query S1 "How many units were affected by recall F-0001-2025?". Strategy: per-record data version in Redis (s9275:ver:recall_notice:<id>). Every cache entry stores the version its answer was built from; the application's write path bumps the version; a hit whose stored version differs is deleted and treated as a miss.

Step	What happened	Answer says	Database	is_cached
1	ask S1	8019	8019 (v0)	True
2	write path: units 8019 → 8500, version 0 → 1		8500 (v1)	
3	ask, VERSION_CHECK off	**8019 — stale hit**	8500	True
4	ask, VERSION_CHECK on: v0 ≠ v1 → entry deleted → agents	8500	8500	False
5	ask again	8500	8500	True
6	restore 8019, version 1 → 2		8019 (v2)	



Screenshot 3.5 — python code/cache_app/stale_demo.py — [3] answer matches database: False, [4] CACHE STALE, new answer 8500

Trade-off: a hit pays one extra Redis GET and no database read, and only answers about the changed record are invalidated. But every write path must bump the version: an UPDATE that bypasses the application (manual SQL, an old Postman request) leaves stale answers in place; answers to general questions are tied to no record and would need a TTL; outdated entries keep using memory until they are hit.

Label note (cache_cases.yaml not edited): P05 "Why was the product in recall F-2015-2026 recalled?" was labeled a paraphrase of S5 (recall category). MiniLM placed it nearest S4 (0.8436), so it counts as a false hit at 0.75 and 0.80. "Why" is wider than "category"; the label stays as committed.

Part 4 — Self-Improving Multi-Agent System

Reference dataset: part4/reference_notices.json — 30 real records from s9275_rel (Random(9275), HW4-seeded rows only), the Evaluator's only ground truth. Inputs (id + question, no answers): training_data.json 10, validation_data.json 10, test_data.json 12. The 30 records are split 10/10/10, so no test question is about a record seen in training or validation. Each file mixes single-field, two-field, unknown recall number and off-domain questions.

reports/hw09/part4 — one reference record and one input item


```
{"id": 4837, "recall_number": "F-4837-2023", "product_name": "Coastal Catch Baby Spinach, 12 oz",
"category": "Mislabeling", "recall_date": "2023-05-29", "units_affected": 1203, "firm_name":
"Heartland Farms LLC", "firm_state": "CO"}
{"id": "train_05", "question": "Which firm issued recall F-2764-2022, and how many units were
affected?"}
```


Agent code

code/self_improve/answerer.py — AnsweringAgent.answer()

```
    def answer(self, question: str, lessons: Optional[list[dict]] = None,
               previous: Optional[str] = None, feedback: Optional[str] = None,
               drop_field: Optional[str] = None) -> dict:
        t0 = time.perf_counter()
        number, record = self.lookup(question)
        shown = dict(record) if record else None
        if shown and drop_field and previous is None:
            shown.pop(drop_field, None)                # controlled incomplete first attempt

        parts = [f"Question: {question}"]
```

```
        if number is None:
            parts.append("Database lookup: the question contains no recall number.")
        elif shown is None:
            parts.append(f"Database lookup: no recall notice {number} in the database.")
        else:
            parts.append("Database record:\n" + "\n".join(f"- {k}: {v}" for k, v in
shown.items()))
        if previous is not None:
            parts.append(f"Your previous answer:\n{previous}\n\nEvaluator
feedback:\n{feedback}\n\n"
                         "Write a corrected answer.")
        prompt = "\n\n".join(parts)

        # Lessons go into the system message as rules. First validation run (lessons at the
        # end of the user message): the 3b model loaded lesson_001 and still answered
        # "Jane Austen" - a trailing note was ignored. See results/validate_run1_*.json.
        system = SYSTEM
        if lessons:
            system += ("\n\nRules learned from earlier mistakes. Always follow them:\n" +
                       "\n".join(f"- [{x['lesson_id']}] Past mistake: {x['failure_pattern']} "
                                 f"Rule: {x['correction']}" for x in lessons))
        reply = self.client.complete([{"role": "system", "content": system},
                                      {"role": "user", "content": prompt}], max_tokens=200)
        return {"answer": reply.text.strip(), "prompt": prompt, "system": system,
                "features": sorted(self.features(question, number, record)),
                "latency_ms": round((time.perf_counter() - t0) * 1000, 1)}
```

code/self_improve/evaluator.py — evaluate()

```
def evaluate(question: str, answer: str, reference: dict) -> dict:
    numbers = [n.upper() for n in RECALL_NUMBER.findall(question)]

    # --- off-domain: no recall number in the question ---------------------------
    if not numbers:
        # Every sentence must decline or talk about recall notices. Found in training
        # (train_07): "I can only provide information about recall notices. ... boil an egg
        # for about 7-10 minutes" - a decline followed by the answer anyway.
        sentences = [x for x in re.split(r"(?<=[.!?])\s+", answer.strip()) if x]
        answering = [x for x in sentences if not (DECLINE.search(x) or re.search(r"recall", x,
re.I))]
        if DECLINE.search(answer) and not answering:
            return _pass("off_domain", "The answer declines a question outside recall notices.")
        return _revise("off_domain", "answered_off_domain",
                       "The question is not about recall notices, but the answer gives "
                       "information that is not in the reference dataset.",
                       "Do not answer it. Say that you can only answer questions about "
                       "grocery recall notices.")

    record = reference.get(numbers[0])

    # --- recall number that is not in the reference dataset ---------------------
    if record is None:
        extra = re.findall(r"\d+", RECALL_NUMBER.sub(" ", answer).replace(",", ""))
        if NOT_FOUND.search(answer) and not extra:
            return _pass("not_found", f"{numbers[0]} is not in the reference dataset and "
                                      "the answer says it was not found.")
        return _revise("not_found", "fabricated_not_found",
                       f"{numbers[0]} is not in the reference dataset, but the answer "
                       "gives details for it as if it existed.",
                       f"Recall {numbers[0]} does not exist. Say it was not found and do "
                       "not give any product, firm, units or date.")

    # --- a real record: every asked field present, no invented numbers ------------
    fields = asked_fields(question)
    missing = [f for f in fields if not contains_value(f, answer, record[f])]
    invented = invented_numbers(answer, record)
    if not missing and not invented:
        return _pass("record", f"All asked fields ({', '.join(fields)}) match the reference "
                               f"record for {numbers[0]}.")
```

```
    problems, feedback = [], []
    if missing:
        problems.append(f"missing or wrong: {', '.join(missing)}")
        feedback.append(f"The answer does not correctly state {', '.join(missing)}. "
                        "Read that field from the record again and state it.")
    if invented:
        problems.append(f"numbers not in the record: {invented}")
        feedback.append(f"The answer contains numbers that are not in the record "
                        f"({', '.join(invented)}). Remove them.")
    ftype = "missing_field" if missing and len(missing) < len(fields) else "wrong_value"
    return _revise("record", ftype,
                   f"{numbers[0]}: " + "; ".join(problems) + ".", " ".join(feedback))
```

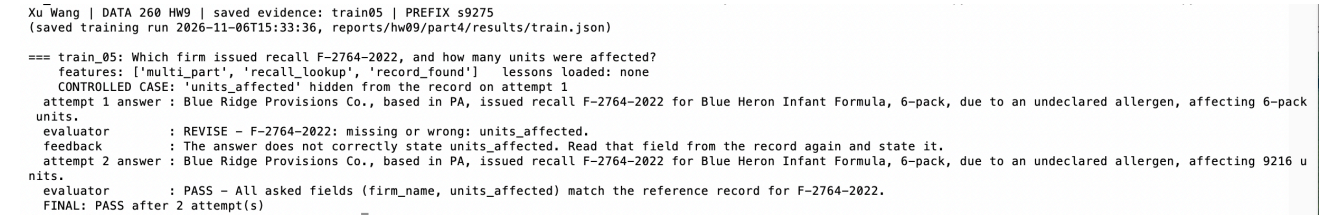
code/self_improve/run_part4.py — run_one(): agent communication and stopping rule

```
def run_one(agent, ref, item, store=None, max_attempts=MAX_ATTEMPTS,
            drop_field=None, learn=False, verbose=False) -> dict:
    q, qid = item["question"], item["id"]
    number, record = agent.lookup(q)
    feats = sorted(agent.features(q, number, record))
    lessons = store.relevant(feats) if store else []
    t0 = time.perf_counter()
    attempts, previous, feedback = [], None, None
    if verbose:
        print(f"\n=== {qid}: {q}")
        print(f"    features: {feats}   lessons loaded: {[x['lesson_id'] for x in lessons] or
'none'}")
        if drop_field:
            print(f"    CONTROLLED CASE: '{drop_field}' hidden from the record on attempt 1")
    for n in range(1, max_attempts + 1):
        out = agent.answer(q, lessons=lessons, previous=previous, feedback=feedback,
                           drop_field=drop_field)
        verdict = evaluate(q, out["answer"], ref)
        attempts.append({"attempt": n, "answer": out["answer"], **verdict,
                         "answer_ms": out["latency_ms"]})
        if verbose:
            print(f"  attempt {n} answer : {out['answer']}")
            print(f"  evaluator        : {verdict['result']} - {verdict['explanation']}")
            if verdict["feedback"]:
                print(f"  feedback         : {verdict['feedback']}")
        if verdict["result"] == "REVISE" and learn and store and not drop_field:
            new = store.record_failure(verdict["failure_type"], qid)
            if new and verbose:
                print(f"  >>> repeated failure '{verdict['failure_type']}' "
                      f"({', '.join(new['created_from'])}) -> NEW LESSON {new['lesson_id']}: "
                      f"{new['correction']}")
        if verdict["result"] == "PASS":
            break
        previous, feedback = out["answer"], verdict["feedback"]
    final = attempts[-1]
    if verbose:
        print(f"  FINAL: {final['result']} after {len(attempts)} attempt(s)")
    return {"id": qid, "question": q, "features": feats,
            "lessons_loaded": [x["lesson_id"] for x in lessons], "attempts": attempts,
            "n_attempts": len(attempts), "final_result": final["result"],
            "final_answer": final["answer"], "question_kind": final["question_kind"],
            "final_failure": final["failure_type"],
            "latency_ms": round((time.perf_counter() - t0) * 1000, 1)}
```

4.1 Repair failed answers using Evaluator feedback

	Controlled test case train_05 (units_affected hidden from the record on attempt 1)
Question	Which firm issued recall F-2764-2022, and how many units were affected?
Initial answer	Blue Ridge Provisions Co., based in PA, issued recall F-2764-2022 for Blue Heron Infant Formula, 6-pack, due to an undeclared allergen, affecting 6-pack units.
Evaluator	REVISE — F-2764-2022: missing or wrong: units_affected.
Feedback	The answer does not correctly state units_affected. Read that field from the record again and state it.

Revised answer	Blue Ridge Provisions Co., based in PA, issued recall F-2764-2022 for Blue Heron Infant Formula, 6-pack, due to an undeclared allergen, affecting 9216 units.
Final evaluation	PASS — All asked fields (firm_name, units_affected) match the reference record for F-2764-2022.
Attempts	2 (maximum 2)
Final response	Blue Ridge Provisions Co., based in PA, issued recall F-2764-2022 for Blue Heron Infant Formula, 6-pack, due to an undeclared allergen, affecting 9216 units.



Screenshot 4.1 — controlled case train_05: attempt 1, evaluator, feedback, attempt 2, FINAL (reprinted from part4/results/train.json)

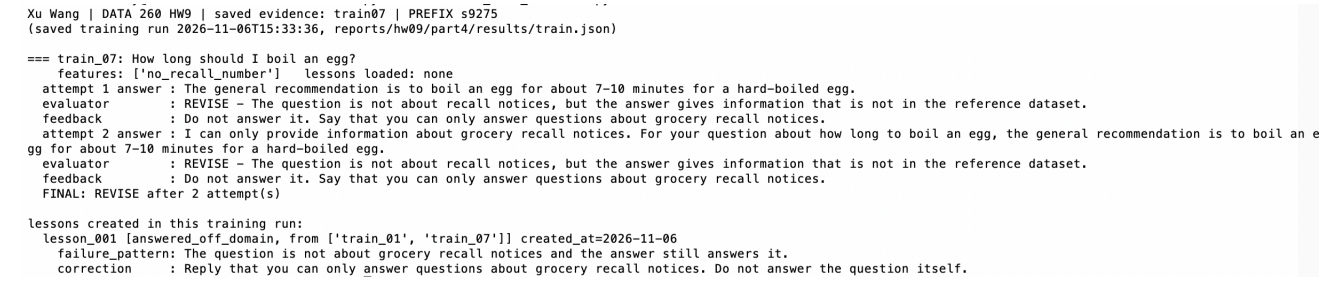
Training result: 9/10 PASS. Four questions failed on attempt 1; feedback fixed three. train_07 ("How long should I boil an egg?") declined and then answered anyway on attempt 2 and stopped as REVISE after the second attempt.

4.2 Save and reuse lessons

A lesson is created when the Evaluator reports the same failure type a second time. Off-domain answering failed on train_01 and again on train_07, so lesson_001 was created; an unknown recall number failed only once (train_04), so no lesson for it. Before answering, the agent loads only lessons whose applies_to matches the question's features, and each load increments use_count.

reports/hw09/part4/lessons.json (required fields + applies_to / status; full validation record in the file)


```
[
 {
  "lesson_id": "lesson_001",
  "failure_pattern": "The question is not about grocery recall notices and the answer still
answers it.",
  "correction": "Reply that you can only answer questions about grocery recall notices. Do not
answer the question itself.",
  "created_at": "2026-11-06",
  "use_count": 6,
  "applies_to": "no_recall_number",
  "status": "kept"
 }
]
```

Screenshot 4.2a — train_07 fails off-domain a second time; lesson_001 created from train_01 + train_07 (reprinted from part4/results/train.json)

Reuse on a later related question: validation val_10 "Who wrote the novel Pride and Prejudice?" loaded ['lesson_001'] into its system message; answer: "I can only answer questions related to grocery recall

notices. Pride and Prejudice was written by Jane Austen.". In the final test, test_10 "What is the best way to learn to play the guitar?" loaded lesson_001 and declined on the first attempt ("I can only answer questions about grocery recall notices. Learning to play the guitar is not related to grocery recall notices."), while without the lesson it stayed wrong after two attempts.

```
Xu Wang | DATA 260 HW9 | saved evidence: reuse | PREFIX s9275
(saved validation run 2026-11-06T15:43:56, test run 2026-11-06T15:51:54)

=== Validation: same question without / with the stored lesson (1 attempt) ===
val_10  "Who wrote the novel Pride and Prejudice?"
  without lesson : Pride and Prejudice was written by the English novelist Jane Austen.  -> REVISE
  loaded ['lesson_001']: I can only answer questions related to grocery recall notices. Pride and Prejudice was written by Jane Austen.  -> REVISE

=== Test (held out): test_10 in each variant ===
Baseline              lessons=[]  attempts=1  REVISE
    final answer: The best way to learn to play the guitar depends on personal learning style, but common methods include taking
Evaluator Repair      lessons=[]  attempts=2  REVISE
    final answer: The best way to learn to play the guitar depends on personal learning style, but common methods include taking
Lesson Memory         lessons=['lesson_001']  attempts=1  PASS
    final answer: I can only answer questions about grocery recall notices. Learning to play the guitar is not related to grocer
Validated Improvement  lessons=['lesson_001']  attempts=1  PASS
    final answer: I can only answer questions about grocery recall notices. Learning to play the guitar is not related to grocer
```

Screenshot 4.2b — lesson_001 (created in training from train_01 + train_07) loaded for later related questions: val_10 in validation and test_10 in the final test (reprinted from saved results)

4.3 Validate improvements before keeping them

validation_data.json is run once without and once with each candidate lesson (one attempt, no repair, so the lesson's own effect is visible). Keep only if failures improve or stay the same, previously correct answers stay correct, and unrelated questions get no new unsupported answers.

Run	Lesson placement	PASS without → with	Regressions	val_10 with the lesson	Decision
1	end of the user message	8/10 → 8/10	none	"…written by Jane Austen." (ignored)	revised
2	system message	9/10 → 9/10	none	declined, then still named Jane Austen	**KEPT**

Run 1 showed the lesson was loaded but ignored by the 3b model, so the lesson was revised to be delivered as a rule in the system message (run 1 kept as results/validate_run1_lesson_ignored.json). Run 2 also includes an Evaluator fix: "does not contain any recall notice" is a correct not-found answer (val_06). lesson_001 met all three rules and was kept; it moved val_10 in the right direction without making it pass. test_data.json was used only after this decision.

```
Xu Wang | DATA 260 HW9 Part 4 | two-agent system | qwen2.5:3b | PREFIX s9275

=== lesson_001 (answered_off_domain) -> KEPT
    validation PASS 9/10 -> 9/10   fixed []   regressions []   unsupported on unrelated 1 -> 1
    ok  failures_improve_or_same
    ok  previously_correct_still_correct
    ok  no_new_unsupported_answers_on_unrelated
    reuse: val_10 "Who wrote the novel Pride and Prejudice?"  loaded ['lesson_001']
           answer: I can only answer questions related to grocery recall notices. Pride and Prejudice was written by Jane Austen.  -> REVISE

lessons.json:
  lesson_001 kept     use_count=2  The question is not about grocery recall notices and the answer still answers it.

saved reports/hw09/part4/results/validate.json
```

Screenshot 4.3 — run_part4.py validate — lesson_001 block: PASS before → after, the three checks, KEPT, lessons.json status

4.4 Four system variants (test_data.json, 12 questions, same file for all runs)

Variant	Correct answers	Incorrect answers	Average attempts	Average latency
Baseline	8	4	1.00	4.52 s
Evaluator Repair	11	1	1.33	4.91 s
Lesson Memory	11	1	1.25	4.71 s
Validated Improvement	11	1	1.25	5.14 s

```
Xu Wang | DATA 260 HW9 | saved evidence: test | PREFIX s9275
(saved test run 2026-11-06T15:51:54, reports/hw09/part4/results/test.json)

=== Part 4.4 results on test_data.json (12 questions) ===
Variant                Correct  Incorrect  Avg attempts  Avg latency
Baseline                     8          4          1.00       4.52 s
Evaluator Repair            11          1          1.33       4.91 s
Lesson Memory               11          1          1.25       4.71 s
Validated Improvement       11          1          1.25       5.14 s
```

Screenshot 4.4— Part 4.4 results table (reprinted from part4/results/test.json)

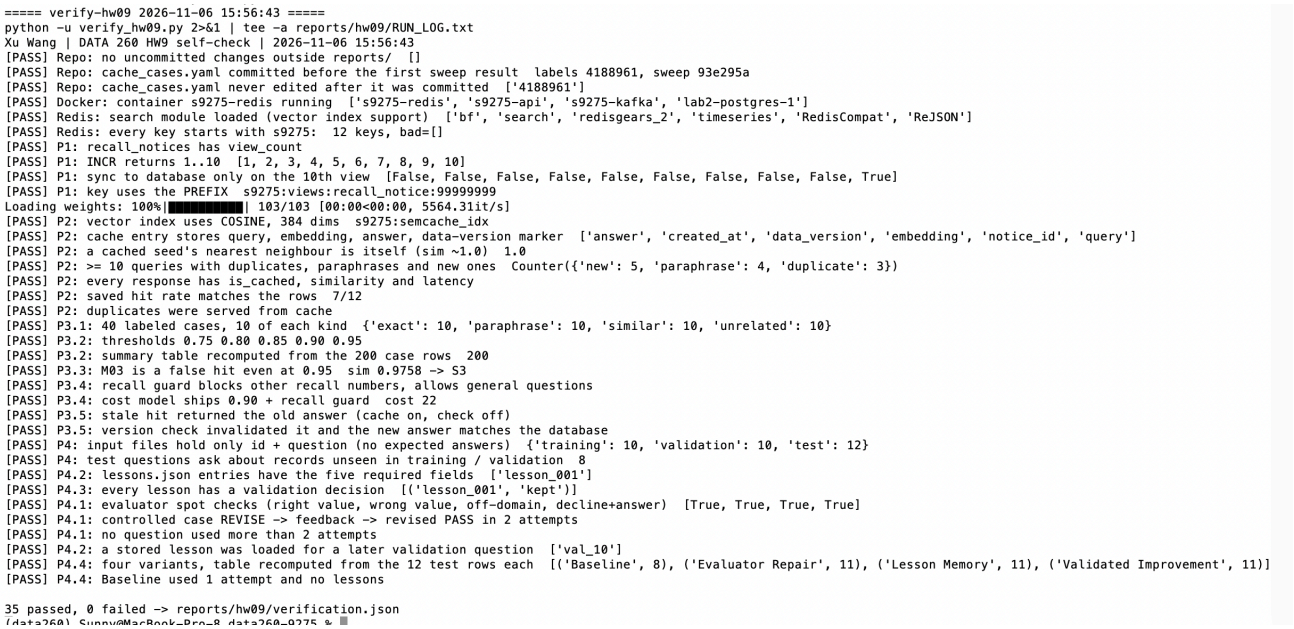
- Baseline fails both off-domain questions and both unknown recall numbers. Evaluator Repair fixes three of the four; test_10 (guitar) repeats the same answer and stays wrong.
- With lesson_001, test_10 declines correctly on the first attempt, so average attempts drop from 1.33 to 1.25.
- Manual review: in both lesson variants test_02 answered "…The question about the number of planets in the solar system is not relevant to this database." — a correct decline that the Evaluator marked REVISE because "not relevant" is not in its decline phrases. The table is the automatic result and was not re-run; by manual review both lesson variants are 12/12.
- Lesson Memory and Validated Improvement ran the same configuration here: validation removed no lesson, so both loaded {lesson_001}. The latency difference is run-to-run noise.

How the system works

- **Agent communication:** The agents never call each other. run_one() passes the question to the Answering Agent, its answer to the Evaluator, and the Evaluator's PASS/REVISE, explanation and feedback back to the Answering Agent as part of the next prompt. The feedback names the field that is wrong, never the value.
- **Lesson reuse:** lessons are created from repeated Evaluator failure types during training, stored in lessons.json, and loaded by relevance (applies_to vs the question's features) into the system message before answering.
- **Stopping condition:** stop at PASS or after the second answer, whatever the second verdict is.
- **Validation:** each candidate lesson is judged on validation_data.json with and without it (three rules above); only kept lessons are loaded by Validated Improvement; the test set never changes the system.
- **Limitation:** the deterministic Evaluator is only as good as its phrase lists. It passed a decline-then-answer in training (fixed), rejected a correct not-found answer in validation (fixed) and rejected a correct decline in the test run (reported, not fixed). A 3b model also follows lessons only partly: it learned to decline but sometimes still answers afterwards.

Verification

make verify-hw09 runs verify_hw09.py and writes reports/hw09/verification.json: 35 passed, 0 failed (commit a90be28). It checks the repo state and label-before-sweep order from git history, Redis keys and the vector index, the Part 1 counter live, and recomputes the Part 2 hit rate, the Part 3 threshold table and the Part 4.4 table from the saved rows.



Screenshot V — make verify-hw09 — final lines with the passed / failed count

AI Use

1. What I used an AI assistant for, and what I did myself.

I used Claude (Anthropic) as a coding assistant.

For each part, Claude proposed a design (where the cache sits in front of the HW8 pipeline, a frozen-cache sweep with one real agent run per case, a per-record version number for invalidation, a deterministic Evaluator and a validation gate for lessons); I reviewed each proposal and approved it before any code was written.

Claude drafted cache_cases.yaml from the database records I pulled, and I wrote METRICS.md and this report.

I decided the labels in cache_cases.yaml before any sweep ran, accepted the cost model (miss = 1, false hit = 20) and the shipped threshold, ran every command on my Mac (Redis, MySQL, Kafka and the three agent terminals, every demo, sweep and Part 4 run), took every screenshot, and checked each result before committing.

2. One AI-produced output that was wrong.

The first version of the Evaluator Agent (written by Claude) passed an answer that should have failed. For the off-domain training question train_07 "How long should I boil an egg?", the revised answer was: "I can only provide information about grocery recall notices. For your question about how long to boil an egg, the general recommendation is to boil an egg for about 7-10 minutes." The rule only checked that a decline phrase appeared somewhere, so it returned PASS — the agent declined and answered anyway, and the Evaluator rewarded it. Because lessons are built from the Evaluator's verdicts, this would have taught the system that "decline, then answer" is correct.

3. How I detected it.

After the first training run I could not find the NEW LESSON line in the summary and asked about it. To check, Claude printed every attempt from results/train.json instead of the summary, and the attempt-2 text

for train_07 still contained the boiling time while its verdict was PASS. I confirmed it by reading the answer myself. (That run is kept as part4/results/train_run1_evaluator_bug.json.)

4. What I changed and why it works now.

The off-domain rule now checks every sentence: each one must either decline or be about recall notices; any other sentence means the question was answered. Two new self-test cases cover it (decline + answer -> REVISE; decline + "unrelated to that topic" -> PASS), and verify_hw09.py re-checks the decline-then-answer case. Re-running training on the same data gave train_07 REVISE after two attempts, which is the correct result, and the failure count that created lesson_001 did not change.

The Evaluator was wrong twice more, in the other direction: in validation it rejected a correct not-found answer ("does not contain any recall notice", fixed before the test run), and in the test run it rejected a correct decline ("not relevant to this database"). The test result was not used to change the system; it is reported as measured, with the manual review written next to the table in METRICS.md.